JS

Fetch API & Async/Await

Week 6 -- Friday -- Web Programming -- Gachon University

By the end of this class you will load real internet data into your own webpage.

`fetch()` `Promise` `.then()` `async` `await` `try/catch`

WEEK 6 -- FRIDAY

Today's Plan

1 **See it first**

Run a live demo BEFORE any explanation

2 **What is fetch?**

The problem it solves + the journey

3 **Promises**

What fetch() actually returns

4 **async / await**

The two magic keywords

5 **Errors & res.ok**

What 404 really does to fetch()

6 **Build it yourself**

Step-by-step joke app + assignment

First: see what you will build today



What just happened?

1. You clicked a button
2. JS sent fetch() to the internet
3. A server returned a joke as data
4. JS put it on the page
5. Page never reloaded!

The problem fetch() solves

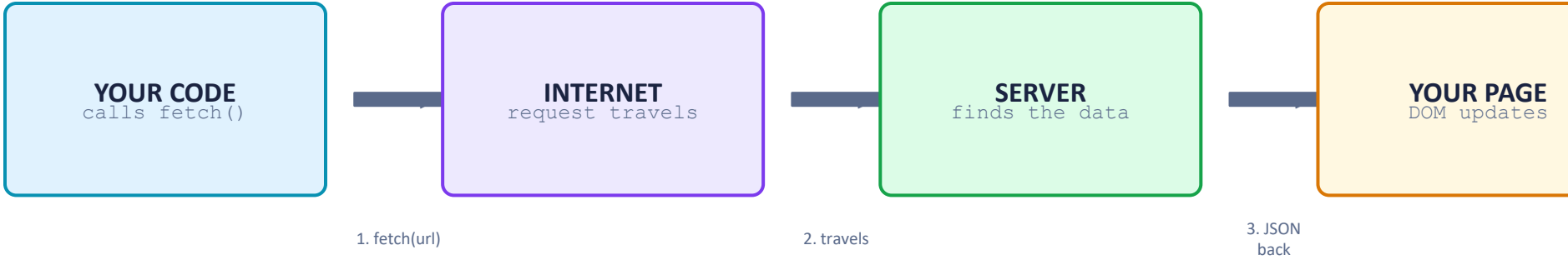
Before fetch() -- Old Way

- x User clicks "Load more"
- x Entire HTML page reloads
- x White screen flash
- x All page state is lost
- x User waits 1-3 seconds
- x Scroll position resets

After fetch() -- New Way

- v User clicks "Load more"
- v JS sends fetch() in background
- v Page stays alive, no flash
- v Scroll position kept
- v Data arrives in < 1 second
- v Only the list updates

What happens when you call fetch()?



Key: While the request is travelling (steps 2-3), your page keeps working normally. Animations run, buttons work, the user can scroll. This is what "asynchronous" means.

Analogy: Like ordering food at a restaurant: you place your order and go back to talking. You do not stand frozen at the counter. The waiter brings food when it is ready.

QUICK CHECK

What does fetch(url) return immediately when you call it?

A The JSON data from the server

B A Promise (a ticket for future data)

C A DOM element

D undefined

ANSWER

A The JSON data from the server

B **A Promise (a ticket for future data)** **v**

C A DOM element

D undefined

Why B? fetch() does not have the data yet -- it just started the request. It immediately returns a Promise: an object that says "I promise to give you the data when it arrives." The data itself comes later.

JavaScript

```
const promise = fetch('https://api.example.com/data');
// promise is returned IMMEDIATELY
// the HTTP request is still in progress in the background
```

A Promise = a cafe ticket



In code:

JavaScript

```
// When promise FULFILLS (coffee ready): .then() runs
fetch(url).then(res => res.json()).then(data => show(data));

// When promise REJECTS (out of oat milk): .catch() runs
fetch(url).catch(err => console.error('Failed:', err));
```

A Promise has exactly 3 states

[?]

PENDING

Request is in progress. JavaScript is waiting for the server to answer.

[v]

FULFILLED

Success! Data arrived. Your .then() callback (or await line) runs next.

[x]

REJECTED

Something failed: no internet, server crashed. Your .catch() or try/catch runs.

Important: A Promise can only move forward: Pending -> Fulfilled OR Pending -> Rejected. It can never go back to Pending once it resolves.

```
.then()  handles Fulfilled    |    .catch()  handles Rejected    |    .finally()  runs ALWAYS
```

QUICK CHECK

A Promise has 3 states.
Which of these is NOT one of them?

A Pending

B Running

C Fulfilled

D Rejected

ANSWER

A Pending

B **Running** v

C Fulfilled

D Rejected

Why B? "Running" is not a Promise state. The three states are: Pending (waiting), Fulfilled (success), and Rejected (failed). There is no "Running" state -- while the request is in progress, the Promise is simply Pending.

PENDING

Request in progress

FULFILLED

Data arrived, .then() runs

REJECTED

Failed, .catch() runs

Writing fetch() -- just 3 things to await

JavaScript

```
async function loadJoke() {

  const res  = await fetch('https://official-joke-api.appspot.com/random_joke');

  const data = await res.json();

  document.getElementById('joke').textContent = data.punchline;

}
```

Step 1: send the request

Step 2: parse the JSON body

Step 3: put it on the page (DOM)

Two awaits: You need await TWICE -- once for the network response, once for parsing JSON. Both steps are asynchronous.

Why TWO awaits?

Await #1 -- The response

```
await fetch(url)
```

Waits for the server to send back the HTTP response. The response is like an envelope -- it is here but not opened yet.

-->

Await #2 -- The data

```
await res.json()
```

Opens the envelope and reads the JSON text, converting it to a JS object. Also async -- so also needs await.

After await #2, "data" is a normal JavaScript object:

JavaScript

```
// data is now a JS object you can read like any other:
data.setup       // --> "Why do programmers prefer dark mode?"
data.punchline   // --> "Because light attracts bugs!"
data.type        // --> "general"
data.id          // --> 42
```

QUICK CHECK

You want to use await inside a function. Where do you put the word async?

A Inside the function, before the first await

B Before the function keyword: async function myFn()

C After the closing brace of the function

D You do not need async -- await works anywhere

ANSWER

A Inside the function, before the first await

B **Before the function keyword: async function myFn() v**

C After the closing brace of the function

D You do not need async -- await works anywhere

Why B? The async keyword goes BEFORE the function declaration. Without async, using await causes a SyntaxError. Think of async as a label that says "this function is allowed to wait for the internet."

JavaScript

```
// CORRECT -- async before the keyword "function"
async function loadData() {
  const res  = await fetch(url);
  const data = await res.json();
  render(data);
}

// ALSO CORRECT -- async arrow function
const loadData = async () => {
  const res = await fetch(url);
```

Two styles -- identical result

.then() chains -- old style

JavaScript

```
fetch('/api/joke')
  .then(res => {
    return res.json();
  })
  .then(data => {
    show(data);
  })
  .catch(err => {
    error(err);
  });
```

=

async/await -- modern style [prefer this]

JavaScript

```
async function getJoke() {
  try {
    const res  = await fetch(
                   '/api/joke');
    const data = await res.json();
    show(data);
  } catch (err) {
    error(err);
  }
}
```

WARNING -- The most common beginner mistake

404 does NOT throw an error!

fetch() only REJECTS when your internet is completely down.

A 404, 500, or any server error STILL RESOLVES the Promise.

Situation	What fetch() does	How to detect
No internet / DNS fails	fetch() REJECTS	catch() runs automatically
Server returns 404	fetch() RESOLVES	YOU must check res.ok
Server returns 500	fetch() RESOLVES	YOU must check res.ok

Always check res.ok

JavaScript

```
async function loadData(url) {
  const res = await fetch(url);

  // res.ok is TRUE  for HTTP 200 - 299  (success)
  // res.ok is FALSE for HTTP 404, 500  (error)
  if (!res.ok) {
    throw new Error('HTTP error: ' + res.status);
  }

  const data = await res.json();
  return data;
}
```

res.ok is TRUE

HTTP status 200 to 299
Request succeeded!
Data is ready.

res.ok is FALSE

HTTP status 400, 404, 500...
Something went wrong.
You must throw yourself!

try / catch / finally -- protect your code

TRY

Your main code goes here.

If ANYTHING inside try throws an error, execution jumps immediately to catch.

CATCH (err)

Runs only when try had an error.

err.message tells you what went wrong.

Show a friendly error to the user here.

FINALLY

ALWAYS runs.
Success OR failure.

Perfect for:
- hiding spinners
- re-enabling buttons
- releasing locks

JavaScript

```
try {
  const res = await fetch(url);
  if (!res.ok) throw new Error('HTTP ' + res.status);
  render(await res.json());
} catch (err) {
  showError(err.message);
} finally {
  hideSpinner();   // always runs
}
```

The complete pattern -- memorise this

JavaScript

```
async function loadData(url) {
  showSpinner();              // tell user something is loading
  try {
    const res  = await fetch(url);
    if (!res.ok) throw new Error('HTTP ' + res.status);
    const data = await res.json();
    render(data);             // everything worked!
  } catch (err) {
    showError(err.message);   // something failed
  } finally {
    hideSpinner();            // always clean up
  }
}
```

Show loading first

res.ok check

catch handles all errors

finally = always

QUICK CHECK

You have a try/catch/finally block. When does finally run?

A Only when try succeeds

B Only when catch runs

C After try AND after catch -- always

D Never -- it is optional

ANSWER

A Only when try succeeds

B Only when catch runs

C **After try AND after catch -- always** **v**

D Never -- it is optional

Why C? finally always runs -- after try OR after catch. This makes it ideal for cleanup: hiding the spinner, re-enabling the "Load" button. If you only put hideSpinner() in try, it won't run when catch handles an error. If you put it in both -- that's code duplication. finally solves this perfectly.

Analogy: Like closing a restaurant after your shift: whether it was a great night (try) or a disaster (catch), you ALWAYS lock the door before going home (finally).

JavaScript

```
// WITHOUT finally -- buggy!
try { data = await fetch(url); hideSpinner(); }
catch (e) { showError(e); }   // spinner stays forever!

// WITH finally -- correct!
try { data = await fetch(url); } catch (e) { showError(e); }
```

Free APIs -- no signup, use right now

Official Joke API

`official-joke-api.appspot.com/random_joke`

Returns setup + punchline. Perfect for this week!

JSONPlaceholder

`jsonplaceholder.typicode.com/posts`

Fake blog posts, users, todos -- built for practice

Open-Meteo

`api.open-meteo.com/v1/forecast?latitude=37.5&...`

Real Seoul weather, no API key needed

Random User

`randomuser.me/api`

Fake profiles with photos, names, emails

PokéAPI

`pokeapi.co/api/v2/pokemon/pikachu`

Full Pokemon database -- sprites, stats, types

REST Countries

`restcountries.com/v3.1/name/korea`

Country flags, population, capital -- good table project

This week: Random Joke Fetcher

Required	A button that calls loadJoke() when clicked
Required	Show "Loading..." while the request runs
Required	Display setup and punchline in separate elements
Required	Error handling with a friendly message
Challenge	Add a loading spinner animation
Extra	Save Favourite button -- use localStorage

API: `official-joke-api.appspot.com/random_joke`

Submit your GitHub Pages link via the Week 6 homework form.

QUICK CHECK

You call fetch() and the server returns HTTP 404. What happens?

A fetch() rejects -- catch() runs automatically

B fetch() resolves -- you must check res.ok yourself

C fetch() returns null

D fetch() throws a SyntaxError

ANSWER

A fetch() rejects -- catch() runs automatically

B **fetch() resolves -- you must check res.ok yourself v**

C fetch() returns null

D fetch() throws a SyntaxError

Why B? fetch() only rejects on true network failure (no internet, DNS error). A 404 from the server is still a valid HTTP response -- fetch() receives it successfully. res.ok will be false. You MUST write: if (!res.ok) throw new Error(...).

JavaScript

```
const res = await fetch(url);

// Without this check, a 404 would silently pass through!
if (!res.ok) throw new Error('HTTP ' + res.status);

const data = await res.json();   // now safe
```

WEEK 6 REVIEW -- 6 things to remember

`fetch(url)`

Sends HTTP GET, returns a Promise immediately

`await`

Pauses THIS function -- not the whole page

`res.json()`

Parses response body -- also needs await

`res.ok`

false for 404/500 -- check it yourself!

`async function`

Required to use await inside any function

`finally`

Always runs -- use for cleanup (spinners, etc.)